
CS164 Final

Programming Languages and Compilers

Fall 2024

Dec 19, 2024

- Please read all instructions (including these) carefully.
- **This is a closed-book exam.** You are allowed no other resources except two handwritten, double-sided (4 sides) cheat sheets.
- Write your name, Berkeley email address, and SID.
- There are **24 pages** and three parts to this exam. If you get stuck on a question, move on and return to it later. You will have **2 hours** to work on the exam.
- Please write your answers in the space provided on the exam and mark your solutions. **Do not** use any additional scratch paper.
- For multiple-choice questions, **fill in the entire bubble.**
If the question allows you to select multiple answers, the correct answer always selects at least one bubble. We will grade no selection as 0 points. Points will be deducted for extraneous selections, but you cannot receive below a zero for each question.
- No electronic devices, including cell phones, are allowed. Turn off all devices and place them in your bag.

Berkeley Email: _____

Name: _____

Student ID: _____

Problem	Max points
1 Potpourri	23
2 Code Generation and Optimization	
3 Language Constructs	
Total	

Part 1: Potpourri

Question 1: 3 points

Which of the following regular expressions match the language of **nonempty** strings over the alphabet $\Sigma = \{a, b\}$ with no adjacent symbols the same? The precedence of the regex operator from lowest to highest is $|$; then concatenation; then $*$, $?$, and $+$.

(Select **all** that apply.)

- ☒ $a \mid b \mid (ab)^+a? \mid (ba)^+b?$
- ☐ $a \mid b \mid (ab)^*a? \mid (ba)^*b?$
- ☒ $((ab)^*ab?) \mid ((ba)^*ba?) \mid a \mid b$
- ☐ $a \mid (ab)^*a? \mid (ba)^*b?$
- ☐ $((ab)^*ab?) \mid ((ba)^*ba)$
- ☒ $a(ba)^*b? \mid b(ab)^*a?$
- ☐ $a(ba)^*b+ \mid b(ab)^*a+$

Question 2: 3 points

If we know a grammar is LR(1), which of the following statements must be true? Select all **true** statements.

- ☐ The grammar is LL(1).
- ☐ The grammar is LL(k), where $k > 1$.
- ☒ The grammar is LR(k), where $k > 1$.
- ☒ The grammar is unambiguous.
- ☐ The grammar is LR(0).
- ☐ The grammar is LALR(1).
- ☒ The grammar's LR(1) parse table has no shift-reduce conflicts.

Question 3: 3 points

Consider the following context-free grammar G , where “ x ” and “ y ” are terminals:

$$E \rightarrow E E \mid y E \mid x$$

Select **all** that apply.

- ☐ There exists exactly one distinct parse tree for the string “ $yxyx$ ”
- There exists exactly two distinct parse trees for the string “ $yxyx$ ”
- ☐ There exists exactly three distinct parse trees for the string “ $yxyx$ ”
- The language generated by G is exactly the language generated by the regex $(y^*x)^+$
- ☐ The language generated by G is exactly the language generated by the regex $(x \mid y)^*$
- The grammar is ambiguous

Question 4: 3 points

Consider the following grammar over the alphabet $\{0, 1, 2, 3\}$:

$$\begin{aligned} S &\rightarrow B A \mid 2 A B \\ A &\rightarrow 1 A \mid 3 B \mid \text{epsilon} \\ B &\rightarrow C B \mid \text{epsilon} \\ C &\rightarrow 1 C \mid \emptyset \end{aligned}$$

Select **all true** statements.

- $\text{First}(A) = \{\text{epsilon}, 1, 3\}$
- ☐ $\text{First}(B) = \{1, \text{epsilon}\}$
- $\text{Follow}(B) = \{\$, 0, 1, 3\}$
- ☐ $\text{Follow}(C) = \{\emptyset\}$
- A LL(1) parser generator would find a conflict for A on lookahead 1
- ☐ A LL(1) parser generator would find a conflict for A on lookahead 3
- ☐ The grammar is LL(1)

Question 5: 6 points

Suppose we extend the ChocoPy language with the following two constructs:

Union Type	$A \mid B$
Split Expression	<code>split e if x: A then e_1 else y: B then e_2</code>

A union type $A \mid B$ is a type that holds a value either of dynamic type A or type B .¹

The split expression evaluates e , an expression of type $A \mid B$ to a value. Then, depending on the value's dynamic type, evaluates to the appropriate case: if the value has dynamic type A , then the split expression evaluates to e_1 ; otherwise, if the value has dynamic type B , then the split expression evaluates to e_2 .

Here is an example program that prints 1:

```
i_or_s: int | str = 1
split i_or_s if x: int then print(x) else y: str then print(y)
```

A) Select **all** of the **sound** subtyping relationships (3 points):

- ☐ $\text{object} \leq (A \mid B)$
- ☒ $(A \mid B) \leq \text{object}$
- ☒ $A \leq (A \mid B)$
- ☐ ~~$(\text{object} \mid B) \leq \text{object}$~~
- ☐ $(A \mid B)$ is not a subtype of $(B \mid A)$
- ☒ $(A \mid B) \leq (B \mid A)$
- ☐ $(A \mid B) \leq A$

¹ For this question, assume that A and B are disjoint (for example, assume $\text{int} \mid (\text{int} \mid \text{str})$ is not a valid union type for this question, since int and $\text{int} \mid \text{str}$ are not disjoint). Also, assume that the value inspected by `split` (that is, the result of e) is not `None`.

We ignored the fourth selection for the purposes of grading. We intended the fourth answer to be sound, but during the exam, we clarified to assume that $(\text{object} \mid B)$ were disjoint even though there is no choice of B that makes the two disjoint.

After the clarification, the fourth answer then should not be selected because B may be inhabited by some non-object value, which would not be valid to assign into an object. This makes the subtyping relationship not sound. We did not intend to change the correct answer with our clarification. Due to general ambiguity in this option we decided to ignore it.

The reason why it is important for A and B to be disjoint in $(A \mid B)$ is because otherwise the semantics of `split` are not easily defined. For example, what is the result of

```
i: (int | int) = 1
split i if x: int then print("a") else y: int then print("b")
```

Similarly, how should `split` on `None` work when both A and B are nullable? When A and B are not disjoint, it is not always possible to uniquely pick a branch of the split, and we did not want to define tie breaking rules as they could be subtle with nested union types.

Finally, this behavior is the difference between *sum types* and *union types*. Sum types conventionally refer to disjoint sum, so it is always possible to distinguish exactly one case, as opposed to union types, which may have overlaps.

B) Complete the rule that type checks a **split** expression. Select the **minimal** set of premises so that the rule is sound and matches the specification (3 points).

- ☒ $O, M, C \vdash e : T$
- ☐ $O, M, C \vdash e : \text{object}$
- ☒ $T \leq_a A \mid B$
- ☐ $T \leq_a \text{object} \mid A$
- ☒ $O[x / A], M, C \vdash e_1 : T_1$
- ☐ $O[x / A], M, C \vdash e_1 : A$
- ☒ $O[y / B], M, C \vdash e_2 : T_2$
- ☐ $O[y / B], M, C \vdash e_2 : B$

$O, M, C, R \vdash \text{split } e \text{ if } x: A \text{ then } e_1 \text{ else } y: B \text{ then } e_2 : T_1 \mid T_2$

Question 6: 3 points

Consider the following grammar:

$S \rightarrow A$
 $A \rightarrow A \ 0 \ 1 \ A \ 1$
 $A \rightarrow B \ 3$
 $B \rightarrow B \ 2$

Recall that the initial state of the LR(1) parsing DFA is the closure of the starting LR(1) item: $S \rightarrow \bullet A, \$$. For each multiple-choice response, select it if that **exact** item belongs in the initial state of the LR(1) parsing DFA.

- ☒ $S \rightarrow \bullet A, \$$
- ☐ $A \rightarrow \bullet B \ 3, \ 0/2/\$$
- ☐ $B \rightarrow \bullet B \ 2, \ 3$
- ☒ $A \rightarrow \bullet B \ 3, \ 0/\$$
- ☒ $A \rightarrow \bullet A \ 0 \ 1 \ A \ 1, \ 0/\$$
- ☐ $B \rightarrow \bullet B \ 2, \ 2$
- ☐ $A \rightarrow \bullet A \ 0 \ 1 \ A \ 1, \ 0/\$/2/3$
- ☐ $A \rightarrow \bullet B \ 3, \ \$$

Common mistakes: selecting both $B \rightarrow \bullet B \ 2, \ 3$ and $B \rightarrow \bullet B \ 2, \ 3$. The wording of the question said to only select a question if the exact item belongs in the initial state, and it also defines an LR item to include the lookahead symbols.

The exact state that appears is $B \rightarrow \bullet B \ 2, \ 2/3$.

Question 7: 2 points

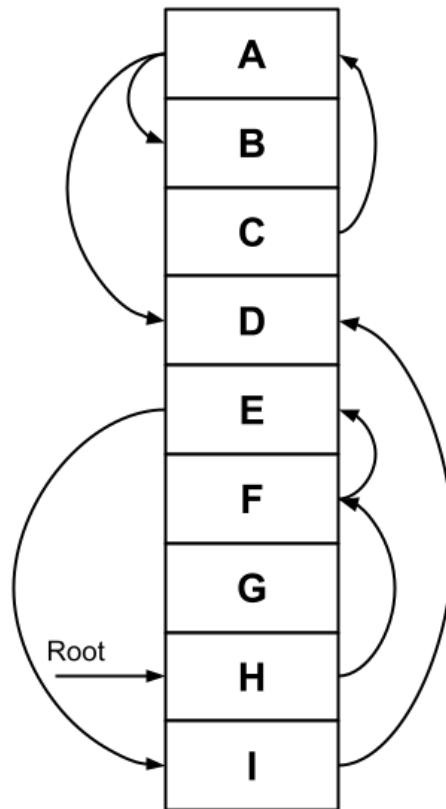
Select **all true** statements.

- ☐ It is not possible for LALR(1) parser generators to encounter reduce/reduce conflicts during parser construction.
- ☒ The set of context-free grammars that are LR(1) are a superset of the set of context-free grammars that are LALR(1).
- ☐ Suppose a LR(1) parser accepts a language L . There always exists an LALR(1) parser that accepts exactly L .
- ☐ Suppose an LR(1) parser generator is able to construct a parser for grammar G without conflict. An LALR(1) parser generator will also be able to construct a parser for G without conflict.

You may use this page as scratch paper. **It will not be graded.**

Part 2: Optimization and Analysis

Question 1: 4 points



Assume the cost of sweeping an object is **1**, the cost of copying an object is **4**, and the cost of all other operations is zero. Fill in the blank with the cost of running each garbage collection algorithm on the heap above.

Mark & Sweep	Stop & Copy
4	20

Question 2: 4 points

Suppose we define the *survival rate* as the percentage of live data remaining after garbage collection. For example, if a 100MB heap contains 15MB of live data after garbage collection, the survival rate is 15%.

Again, assume the cost of sweeping an object is **1**, the cost of copying an object is **4**, with all other costs negligible. For what threshold survival rate **t** is the cost of **Mark & Sweep** equal to the cost of **Stop & Copy**? (Fill in the blank; 2 points)

$$t = \boxed{20} \%$$

Solution: We want to solve $1 \cdot (1-t) = 4 \cdot t \Rightarrow 1 - t = 4t \Rightarrow 1 = 5t \Rightarrow t = 1/5$

Suppose the actual survival rate of a program is **s**. If **s < t**, which algorithm has **less** cost? (Select one; 2 points)

- ☐ Mark & Sweep
- ☒ Stop & Copy

Question 3: 2 points

Suppose a program using **Mark & Sweep** runs garbage collection $n \geq 0$ times before exiting. If we change the program to use **Stop & Copy**, how many times might the new program run garbage collection? Assume we run garbage collection when the program runs out of heap memory and that both algorithms have access to the same size memory pool. Select all possible bounds.

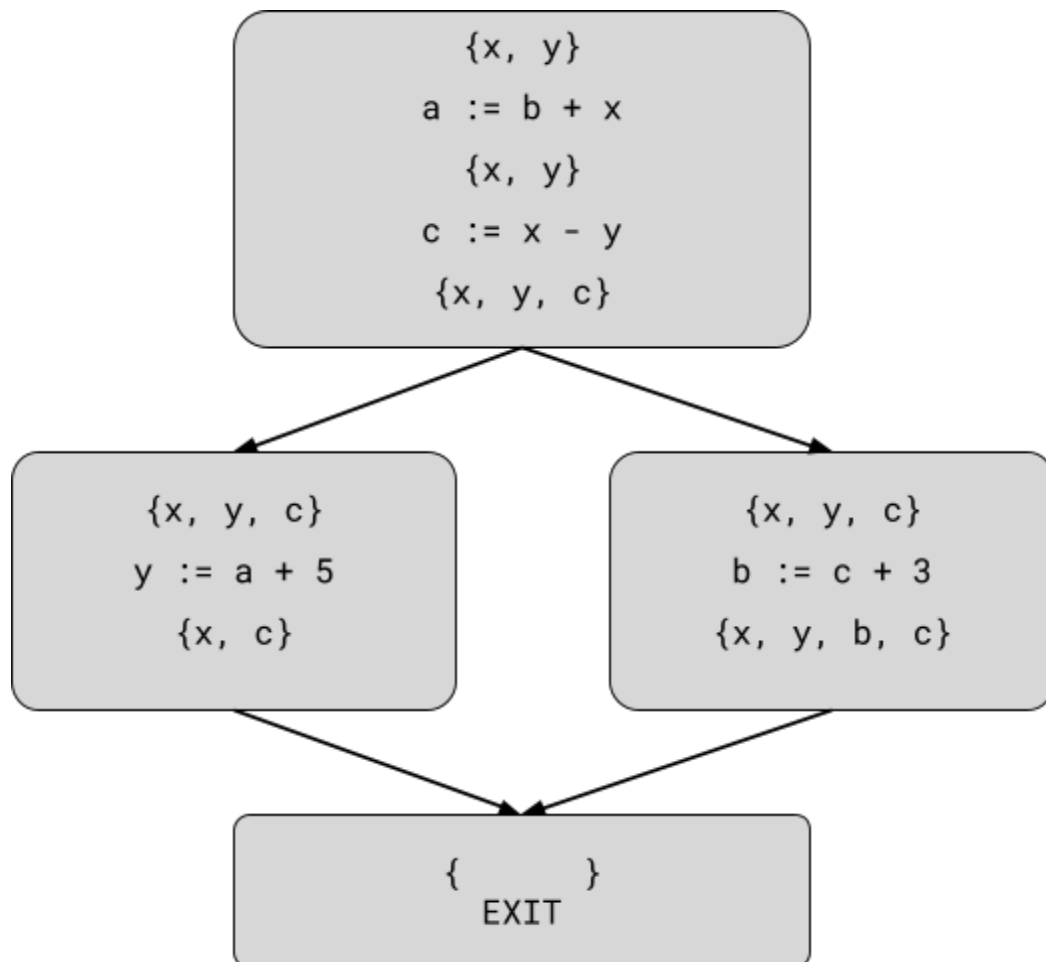
- ☐ $< n$
- ☒ $= n$ (this bound is achieved when $n = 0$)
- ☒ $> n$
- ☐ It is impossible to determine which algorithm will run more frequently, even for the same program.

Question 4: 7 points

Let's design an analysis that computes at each program point the set of variables whose value is guaranteed to depend only on the values of local variables. We call such variables *locally computable variables*. We say that **a variable is locally computable at a program point** if all right-hand side of assignments that reach the variable at that program point contain only the following:

1. constants
2. other locally computable variables
3. arithmetic on locally computable variables

The example below shows a function with a set of global variables $\{a, b, c\}$ and a set of local variables $\{x, y\}$. We've also written the set of locally computable variables at each program point except for the one before EXIT.



A) What is the set of locally computable variables at the program point just before EXIT?
(1 point)

{x, c}

B) What is the top of our analysis domain? Select the answer that computes the most precise analysis. (2 point)

- ☐ S, where S is the set of all local variables in our function
- ☒ S, where S is the set of all global/local variables in the program/function, respectively.
- ☐ S, where S is the set of all local variables and constants in the function
- ☐ S, where S is the set of all global variables in the program

We need top to be /all/ variables; otherwise, we will not discover global variables being locally computable in certain programs with loops. For example, let g be global:

```
g = 1
while cond:
    g = 1
# loop exit: g is clearly locally computable here
```

Clearly, g is locally computable at the loop exit as it can only take on a constant value, but if you run the dataflow with top being just the set of local variables, the lattice meet at the beginning of the loop header will always remove g from the set of locally computable variables, so we will never discover it as being locally computable in the exit of the loop.

Note: the entry point is not necessarily initialized to top. It is initialized to the boundary lattice value, which in this analysis is just the set of all local variables.

C) Complete the blank. (Select all that apply; 2 points)

$\text{Out}(x = v) = \text{-----}$ where v is a variable

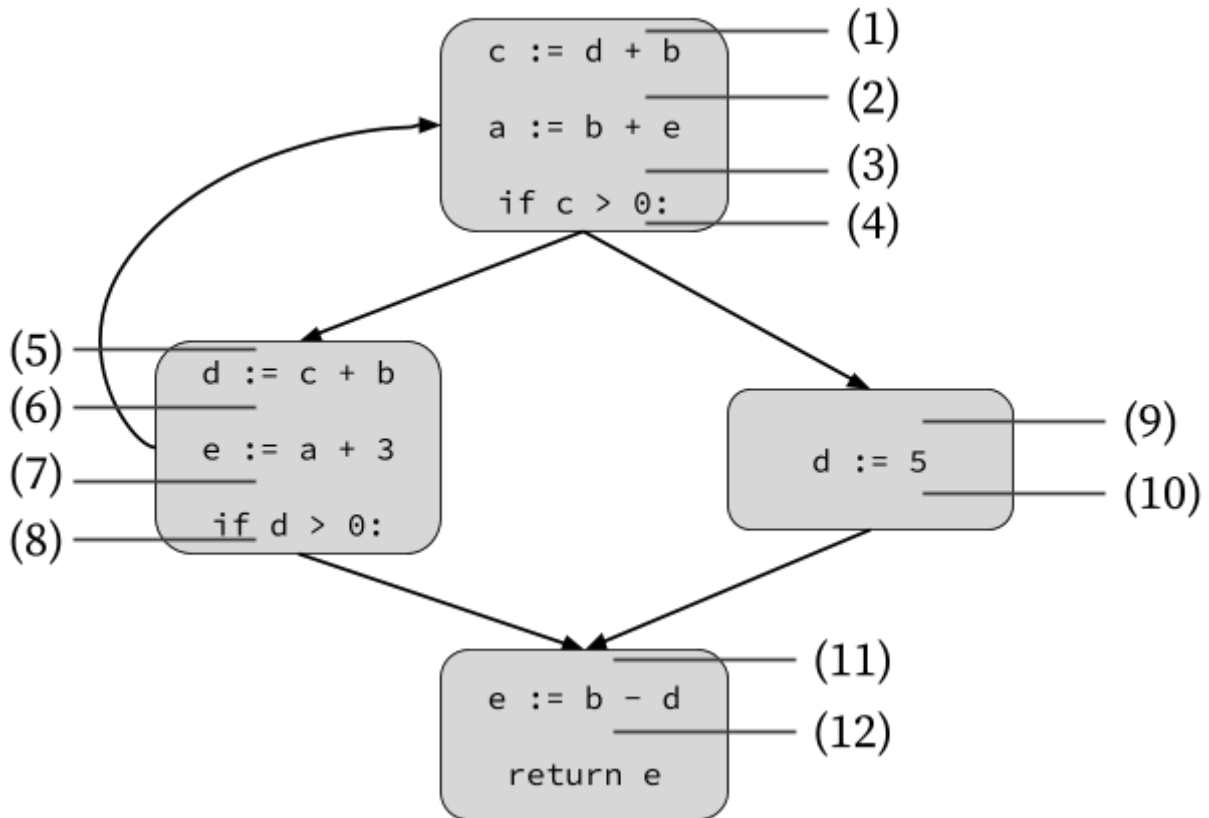
- ☐ $\text{In}(s) - \{v\}$ if $x \notin \text{In}(s)$
- ☐ $\text{In}(s) \cap \{v\}$ if $x \in \text{In}(s)$
- ☐ $\text{In}(s) \cup \{v\}$ if $x \in \text{In}(s)$
- ☒ $\text{In}(s) - \{x\}$ if $v \notin \text{In}(s)$
- ☐ $\text{In}(s) \cap \{x\}$ if $v \in \text{In}(s)$
- ☒ $\text{In}(s) \cup \{x\}$ if $v \in \text{In}(s)$

D) What kind of analysis is this? (Select one; 1 point)

- ☐ Forwards may
- ☒ Forwards must
- ☐ Backwards may
- ☐ Backwards must

Question 5: 6 points

Run a liveness analysis on the following program, filling in each blank with the variables that are live at each program point. Each blank will be graded independently.



(1) {b, d, e}	(2) {b, c, e}	(3) {a, b, c}	(4) {a, b, c}
(5) {a, b, c}	(6) {a, b, d}	(7) {b, d, e}	(8) {b, d, e}
(9) {b}	(10) {b, d}	(11) {b, d}	(12) {e}

Question 6: 6 points

A) Consider the program in **Question 5**. Suppose we decide to spill b to the stack location ba . Write down all locations in the program where we need to add the instruction `store b, ba`. Write **None** if no stores need to be inserted (**1.5 points**).

None

B) Write down all locations in the blank where we need to add `b := load ba`. Write **none** if `b := load ba` is unnecessary to spill b (**1.5 points**).

(1), (2), (5), (11)

C) Return to the original liveness analysis (that is, assume we have not spilled b). For each variable, what are its neighbors in the register interference graph (RIG) (**3 points**)?

Completely mark the appropriate boxes in the adjacency matrix. For example, we have marked the box that says that a interferes with b .

$\downarrow$ <i>interferes with</i> $\rightarrow$	a	b	c	d	e
a		☒	☒	☒	☐
b			☒	☒	☒
c				☐	☒
d					☒
e					

Part 3: Language Constructs

In this section², we will extend ChocoPy to support `foreach` loops, which can loop over *iterators*. Iterators are objects that support a `__next__` method with a particular signature (to be defined below). For example, here is a `foreach` loop that consumes an iterator to print out the integers 0, 1, and 2:

```
class NullableInt(object):
    val: int = 0

    def __init__(self: NullableInt, val: int):
        self.val = val

class RangeIterator(object):
    start: int = 0
    limit: int = 0

    def __init__(self: RangeIterator, start: int, limit: int):
        self.start = start
        self.limit = limit

    def __next__(self: RangeIterator) -> NullableInt:
        if not self.start < self.limit:
            return None
        else:
            self.start = self.start + 1
            return NullableInt(self.start - 1)

i: NullableInt = None
foreach i in RangeIterator(0, 3):
    print(i.val)
```

² For this section please assume the provided ChocoPy snippets compile. This example does not compile; the error is that ChocoPy does not support overriding methods with different parameters. However, the rationale for that restriction does not make sense for `__init__`. Instead, assume Python's behavior: `__init__` receives parameters passed in when creating the new object; for example, `NullableInt(3)` would call `__init__(/* new instance */, 3)`.

Question 1: 4 Points

The way the `foreach var in iter: body` statement works is as follows:

- **Step 1:** the expression `iter` is evaluated to a value that must be an iterator. An iterator is an object that defines the `__next__` method satisfying:
 - The `__next__` method must take no arguments (other than the `self` parameter) and return a value assignable to the declared type of `var`.
 - The return type of `__next__` must be nullable.
- **Step 2:** The value is bound to the (fresh) variable `__it__`.
- **Step 3:** `__it__.__next__()` is evaluated and bound to the variable `var`.
- **Step 4:** If `var` is `None`, then the loop exits. Otherwise, the evaluation proceeds to **Step 5**.
- **Step 5:** `body` is evaluated. Go to **Step 3**.

The `foreach` statement can be expressed equivalently in terms of syntactic constructs already supported by ChocoPy. Fill in the blanks to produce a ChocoPy program with the same output as the example on the previous page. Assume the `Integer` and `RangeIterator` classes have already been defined.

```
var: NullableInt = None
```

```
__it__: RangeIterator = None
```

```
__it__ = RangeIterator(0, 3)
```

```
i = __it__.__next__()
```

```
while not i is None :
```

```
    print(i.val)
```

```
    i = __it__.__next__()
```

Question 2: 6 points

Complete the rule that type checks a **foreach** statement. Select the **minimal** set of premises so that the rule is sound and matches the specification listed in **Question 1**.

- ☐ $O, M, C, R \vdash \textit{iter}$
- ☐ $O, M, C, R \vdash \textit{iter} : T_1$
- ☒ $O, M, C, R \vdash \textit{iter}.__\textit{next}__() : T_1$
- ☐ $O(\textit{var}) = T_1$
- ☒ $O(\textit{var}) = T_2$
- ☒ $T_1 \leq_a T_2$
- ☐ $T_2 \leq_a T_1$
- ☒ $\textit{None} \leq_a T_1$
- ☐ $\textit{None} \leq_a T_2$
- ☒ $O, M, C, R \vdash \textit{body}$
- ☐ $O, M, C, R \vdash \textit{body} : R$
- ☐ $O[T_2 / \textit{var}], M, C, R \vdash \textit{body}$

$O, M, C, R \vdash \textit{foreach var in iter: body}$

Question 3: 6 points

Complete the rule that defines the operational semantics of a **foreach** statement. Select the **minimal** set of premises so that the rule is sound and matches the specification listed in **Question 1**. Assume `__it__` is a fresh (i.e. otherwise unused) variable.

There was a bug in this question; instead of the premise

$$\blacksquare \quad G, E, S_3 \vdash \textit{body} : _, S_4, _$$

we should have given the premises

$$\blacksquare \quad E(\textit{var}) = l_{\textit{var}}$$

$$\blacksquare \quad G, E, S_3[v_{\textit{var}} / l_{\textit{var}}] \vdash \textit{body} : _, S_4, _$$

Which represent binding the result of `__it__.__next__()` to the variable *var* for the body of the for loop. We awarded full credit to the intended solution below because it is close enough to the real solution.

$$\square \quad G, E, S_0 \vdash \textit{iter} : _, S_1, R$$

$$\blacksquare \quad G, E, S_0 \vdash \textit{iter} : v_{\textit{iter}}, S_1, _$$

$$\blacksquare \quad E(\textit{__it__}) = l_{\textit{__it__}}$$

$$\blacksquare \quad S_2 = S_1[v_{\textit{iter}} / l_{\textit{__it__}}]$$

$$\square \quad S_2 = S_1$$

$$\square \quad G, E, S_2 \vdash \textit{iter}.\textit{__next__}() : v_{\textit{var}}, S_3, _$$

$$\blacksquare \quad G, E, S_2 \vdash \textit{__it__}.\textit{__next__}() : v_{\textit{var}}, S_3, _$$

when $v_{\textit{var}}$ has value `None`

$$\square \quad G, E, S_3 \vdash \textit{body} : _, S_4, _$$

$$\square \quad G, E, S_4 \vdash \textit{foreach var in iter: body} : _, S_5, R$$

$$\square \quad G, E, S_4 \vdash \textit{foreach var in __it__: body} : _, S_5, R$$

$$\blacksquare \quad S_5 = S_3$$

otherwise

- $G, E, S_3 \vdash \text{body} : _, S_4, _$
 - $G, E, S_4 \vdash \text{foreach var in iter: body} : _, S_5, R$
 - $G, E, S_4 \vdash \text{foreach var in __it__: body} : _, S_5, R$
 - $S_5 = S_3$
-

$G, E, S_0 \vdash \text{foreach var in iter: body} : \underline{\hspace{4cm}}$

$G, E, S_0 \vdash \text{foreach var in iter: body} : \underline{\hspace{1cm}}, S_5, R \underline{\hspace{1cm}}.$

Iterables

We want to extend the `foreach` statement to handle *iterables*. An iterable is an object that defines a method `__iter__` that returns an iterator. When `__iter__` is called, a fresh iterator is returned each time, meaning iterables can be iterated over multiple times. (For example, in Python, lists and dicts are **iterables**, not iterators.)

If the `foreach` statement is invoked on an iterable, it calls the `__iter__` method to get an iterator, and then loops over the returned iterator as defined in **Question 1**.

For example, the following program defines a `Range` object, which returns a fresh `RangeIterator` each time it is used in a `foreach` loop. The output of this program is the integers `0, 1, 2, 0, 1, 2`.

```
class Range(object):
    start: int
    limit: int

    def __init__(self: Range, start: int, limit: int):
        self.start = start
        self.limit = limit

    def __iter__(self: Range) -> RangeIterator:
        return RangeIterator(start, limit)

r: Range = None
i: NullableInt = None
r = Range(0, 3)

foreach i in r:
    print(i)

foreach i in r:
    print(i)
```

Question 4: 2 Points

Complete the rule that type checks a **foreach** statement over an iterable, in terms of the rule that type checks a **foreach** statement over an iterator. Your answer should have **exactly two premises** and satisfy the specification on the previous page. (Answers with more than two premises or other quantifiers will receive no credit.)

$O, M, C, R \vdash \text{iterable}. \underline{\hspace{10em}} : T$

$O, M, C, R \vdash \text{foreach} \underline{\hspace{10em}} : \text{body}$

$O, M, C, R \vdash \text{foreach var in iterable: body}$

Question 5: 1 Points

Complete the rule that defines the operational semantics of a **foreach** statement over an iterable. You may assume that the program is well-typed. Your answer should have exactly **one premise** (answers with more than one premise or other quantifiers will receive no credit).

$G, E, S_0 \vdash \text{foreach} \underline{\hspace{10em}} : _, S_1, R$

$G, E, S_0 \vdash \text{foreach var in iterable: body} : _, S_1, R$

Question 4: 2 Points

Complete the rule that type checks a **foreach** statement over an iterable, in terms of the rule that type checks a **foreach** statement over an iterator. Your answer should have **exactly two premises** and satisfy the specification on the previous page. (Answers with more than two premises or other quantifiers will receive no credit.)

$O, M, C, R \vdash \text{iterable}. \underline{\text{iter}().\text{next}()} : T$

$O, M, C, R \vdash \text{foreach } \underline{\text{var in iterable. iter}()} : \text{body}$

$O, M, C, R \vdash \text{foreach var in iterable: body}$

Common mistakes:

$O, M, C, R \vdash \text{iterable}. \underline{\text{iter}()} : T$

The specification requires `__iter__` to return an **iterator**. With the above rule, the iterable could return another iterable.

Question 5: 1 Points

Complete the rule that defines the operational semantics of a **foreach** statement over an iterable. You may assume that the program is well-typed. Your answer should have exactly **one premise** (answers with more than one premise or other quantifiers will receive no credit).

$G, E, S_0 \vdash \text{foreach } \underline{\text{var in iterable. iter}()} : _, S_1, R$

$G, E, S_0 \vdash \text{foreach var in iterable: body} : _, S_1, R$

Question 6: 6 Points

Complete the following definition of **cgen** for a stack machine that generates code for a **foreach** loop on an **iterator**.

Assume:

- Variables are allocated an fp-relative slot in the activation record.
- The **head** and **end** labels are unique to the program.
- Booleans are unboxed.
 - The integer **0** represents **False**.
 - The integer **1** represents **True**.

You may find the following definitions useful.

Item	Definition
<code>cgen(node);</code>	Generates code for <code>node</code> . If <code>node</code> is an expression, then <code>cgen</code> leaves the result of the expression in <code>a0</code> .
<code>sw rs2 imm rs1;</code>	Stores the word in <code>rs2</code> to the address <code>rs1 + imm</code> .
<code>beq rs1 rs2 label;</code>	Compares the values in <code>rs1</code> and <code>rs2</code> . If they are equal, branches to <code>label</code> .
<code>bne rs1 rs2 label;</code>	Compares the values in <code>rs1</code> and <code>rs2</code> . If they are not equal, branches to <code>label</code> .
<code>j label;</code>	An unconditional jump to <code>label</code> .
<code>offset(var)</code>	Returns the fp relative offset for the variable <code>var</code> .
<code>__it__</code>	A variable that is guaranteed to be unused anywhere else in the program.

Each blank must contain a single identifier. An identifier is a string containing only numbers, letters, underscores, and parentheses (no spaces or other punctuation). Blanks that do not contain a single identifier will receive zero credit.

```
cgen(foreach var in iterator: body) =
```

```
    cgen( _____ ) ;
```

```
    sw _____ offset(__it__) fp ;
```

```
head:
```

```
    cgen( _____ . _____ ) ;
```

```
    sw _____ offset(var) fp ;
```

```
    cgen(var is None);
```

```
    _____ x0 _____ ;
```

```
    cgen( _____ ) ;
```

```
    _____ ;
```

```
end:
```

```
cgen(foreach var in iterator: body)=
```

```
    cgen( __iterator__ )
```

```
    sw __a0__ offset(__it__) fp ;
```

```
head:
```

```
    cgen( __it__ . __next__() )
```

```
    sw __a0__ offset(var) fp ;
```

```
    cgen(var is None) ;
```

```
    __bne__ __a0__ x0 __end__ ;
```

```
    cgen( __body__ ) ;
```

```
    __j__ __head__ ;
```

```
end:
```

Question 7: 6 Points

ChocoPy does not support higher-order functions, i.e., functions that accept and return other functions. For example, we might want to write:

```
def repeat(times: int, op: (int, int) -> int, num: int) -> int:
    if times <= 1:
        return num
    return op(num, repeat(times - 1, op, num))
```

```
def add(l: int, r: int) -> int:
    return l + r
```

```
def mul(l: int, r: int) -> int:
    return l * r
```

```
four_times_five: int = 0
```

```
two_cubed: int = 0
```

```
four_times_five = repeat(5, add, 4)    # 20
```

```
two_cubed = repeat(3, mul, 2)          # 8
```

A) Suppose that we want to add higher-order functions to ChocoPy with the exact syntax above. Select **all true** statements (4 points):

- ☐ To support higher-order functions, we **must** modify ChocoPy's lexer.
- ☒ To support higher-order functions, we **must** modify ChocoPy's parser.
- ☒ To support higher-order functions, we **must** modify ChocoPy's type checker.
- ☐ To support higher-order functions, we **must** modify a dataflow-based constant propagation optimization.

B) It is possible to use ChocoPy’s object system to emulate passing “functions” as arguments to other functions. What feature of ChocoPy’s object system enables this? Write at most **two** words (**2 points**).

There are multiple valid answers. Please pick **exactly one**; we will accept any one. Responses that have more than one answer or write more than two words will receive zero credit.

+ 2.0: method overriding / dynamic dispatch
+ 1.5: dispatch table / dispatch pointer
+ 1.0: methods / “dispatch”
+ 1.0: inheritance / subclassing / type hierarchy / polymorphism

Rationale for bins:

The main answer we were looking for was method overriding or dynamic dispatch – the idea to emulate higher order functions is to create two objects Add and Mul that inherit from a common Operator class. The common Operator class defines a method run that takes in the two integers which the Add and Mul classes override. Then, calling run would dispatch to the correct method based on the dynamic type of the passed in object.

“Dispatch table” and “dispatch pointer” are *implementation strategies* for dynamic dispatch, but they are not strictly speaking a feature of the object system, so we gave students who answered that a small penalty. We gave “symbol table” no credit because the key word is dispatch.

Both methods and dispatch are necessary, but specifically dynamic dispatch is the feature that enables this. With just static dispatch, you cannot use objects to emulate higher order functions.

Similarly, inheritance / subclassing / a type hierarchy / polymorphism are necessary, but it is possible to have those features without supporting dynamic dispatch (e.g. C has a type “hierarchy” for number types but no dispatch).

Blank Space!

*Feel free to use this space to share feedback on the course or exam or just doodle! This page will not be considered for **grading purposes**.*